CLASE 1 — PRESENCIAL

XML, JSON y Git

Lenguajes Informáticos 1 | **UNAHUR 2025**

Unidad 1 — Lenguajes de marcado y estructura de documentos

catalogo.xml

turnos.json

README.md

¿Qué hacemos hoy? (3 horas)

Hoja de ruta de la clase presencial

20 min

Bienvenida y herramientas

Presentación de la materia. Instalación de VS Code y Git.

20 min

Intro: XML, JSON y Git

¿Para qué sirven? ¿Cuándo usar cada formato? Conceptos de control de versiones.

1 hora

Ejercicio 1: TurismoYa S.A.

Modelado de un catálogo de destinos turísticos en XML.

20 min

Pausa

1 hora y 30 min

Ejercicio 2: MediTrack

Modelado de turnos médicos en JSON. Comparación XML vs JSON.

Opcional

Primer commit del TP

Crear repositorio en GitHub y hacer el primer commit grupal.

¿Qué es un lenguaje de marcado?

La base conceptual de la materia

"Un lenguaje de marcado usa etiquetas para darle estructura y significado al contenido."

🏷️ Las etiquetas

Son marcas que rodean el contenido y le dan significado. Ejemplo: <titulo> le dice al sistema que ese texto es un título.

🌲 La jerarquía

Los elementos se anidan unos dentro de otros, formando una estructura de árbol. Un elemento padre puede tener muchos hijos.

📋 El contenido

El texto real que queremos representar. El lenguaje de marcado envuelve el contenido sin mezclarse con él.

XML — eXtensible Markup Language

Un formato para estructurar e intercambiar datos

📅 Estándar del W3C desde 1998. Todavía ampliamente usado en sistemas empresariales.

🔤 Las etiquetas las define el programador — no hay un conjunto fijo como en HTML.

🔗 Ideal para intercambio de datos entre sistemas. APIs SOAP, configuraciones, exportaciones.

✅ Soporta validación con DTD o XSD: se puede definir un esquema obligatorio de estructura.

Ejemplo

```
<?xml version="1.0"
  encoding="UTF-8"?>

<producto id="P01">
  <nombre>
    Laptop Pro
  </nombre>
  <precio moneda="USD">
    1299.99
  </precio>
  <stock>
    42
  </stock>
</producto>
```

JSON — JavaScript Object Notation

El formato más usado en APIs y aplicaciones web modernas

🌐 Nativo de JavaScript pero usado en todos los lenguajes. Estándar de facto para APIs REST.

⚡ Más liviano que XML: menos caracteres para representar la misma información.

📦 Soporta tipos de datos: string, número, booleano, null, array y objeto.

🔄 Los navegadores lo leen de forma nativa. Ideal para comunicación cliente-servidor.

Ejemplo

```
{
  "id": "P01",
  "nombre":
    "Laptop Pro",
  "precio": {
    "valor": 1299.99,
    "moneda": "USD"
  },
  "stock": 42,
  "disponible": true
}
```

XML vs JSON — ¿Cuándo usar cada uno?

Dos herramientas distintas para distintos problemas

	XML	JSON
Sintaxis	Etiquetas de apertura y cierre	Pares clave-valor: "clave": "valor"
Tipos de datos	Todo es texto. Los tipos se definen con XSD.	Nativo: string, número, bool, null, array, objeto
Verbosidad	Más reiterativo (etiquetas repiten nombre).	Más compacto para la misma información.
Validación	Soporta DTD y XSD — esquema formal y estricto.	JSON Schema (menos extendido y no nativo).
Uso típico	Configs, intercambio entre sistemas, documentos.	APIs REST, apps web/móvil, bases de datos NoSQL.
Soporte browser	No nativo — requiere parser.	Nativo: JSON.parse() y JSON.stringify().

Ejercicio Integrador 1 — TurismoYa S.A.

Modelado de un catálogo de destinos en XML

🏢 **El caso**

TurismoYa S.A. necesita un archivo XML para compartir su catálogo de destinos con agencias socias. Las agencias ya tienen parsers XML compatibles. El archivo debe incluir metadatos y al menos 3 destinos con: nombre, país, precio base en USD, duración en días y descripción.

Consigna

1. Crear el archivo catalogo.xml con la estructura indicada.
2. Usar un elemento raíz <catalogo> con atributos de metadatos (version, fecha, empresa).
3. Cada destino: elemento <destino> con atributo id y subelementos para cada dato.
4. Abrir el archivo en el navegador y verificar que no haya errores de sintaxis.
5. Reflexionar: ¿qué ventajas tiene XML sobre una planilla de cálculo para este caso?

Solución — catalogo.xml

TurismoYa S.A.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<catalogo version="2.1" fecha="2025-04-01"
  empresa="TurismoYa S.A.">

  <destino id="D001">
    <nombre>Cataratas del Iguazú</nombre>
    <pais>Argentina</pais>
    <precio_usd>320</precio_usd>
    <duracion_dias>4</duracion_dias>
    <descripcion>Patrimonio Natural de la Humanidad.</descripcion>
  </destino>

  <destino id="D002">
    <nombre>Torres del Paine</nombre>
    <pais>Chile</pais>
    <precio_usd>580</precio_usd>
    <duracion_dias>7</duracion_dias>
    <descripcion>Trekking en la Patagonia.</descripcion>
  </destino>

  <!-- agregar más destinos aquí -->
</catalogo>
```

Ejercicio Integrador 2 — MediTrack

Modelado de turnos médicos en JSON

🏢 **El caso**

MediTrack es una startup de gestión de turnos médicos. Su API REST devuelve datos en JSON. La diseñadora UX necesita entender la estructura de datos para armar los prototipos. El equipo debe modelar los turnos del día para el consultorio del Dr. Ramírez.

Consigna

1. Crear el archivo turnos.json con los turnos del 1 de abril de 2025.
2. Incluir al menos 3 turnos con distintos estados: pendiente, confirmado y cancelado.
3. Cada turno: número, hora, estado, paciente (nombre + DNI), médico (nombre + especialidad).
4. Un turno debe tener el campo 'observaciones' y otro no — practicar campos opcionales.
5. Validar en jsonlint.com y comparar con el XML del Ejercicio 1.

Solución — turnos.json

MediTrack

```json
{
  "consultorio": "Dr. Ramírez",
  "fecha": "2025-04-01",
  "turnos": [
    {
      "numero": 1, "hora": "09:00",
      "estado": "confirmado",
      "paciente": {
        "nombre": "Ana García", "dni": "31456789"
      },
      "medico": { "nombre": "Dr. Ramírez",
        "especialidad": "Clínica General" },
      "observaciones": "Traer estudios previos."
    },
    { "numero": 2, "hora": "09:30",
      "estado": "pendiente",
      "paciente": { "nombre": "Luis Fernández",
        "dni": "28901234" },
      "medico": { ... }
      // sin campo observaciones
    }
  ]
}
```

Git — Control de versiones

La herramienta que usaremos durante todo el cuatrimestre

¿Por qué Git en esta materia?

👥 El TP es grupal: Git permite que varios trabajen sobre el mismo código sin pisarse.

📸 Cada commit es evidencia de avance: quién hizo qué y cuándo.

🏭 Es la herramienta estándar de la industria. Lo van a usar en cualquier trabajo.

🔴 Los commits en Clases 4, 6, 10 y 12 son obligatorios para rendir el 2.do parcial.

Commits obligatorios:

- **C1** — Clase 4
- **C2** — Clase 6
- **C3** — Clase 10
- **C4** — Clase 12

Git — Flujo de trabajo y comandos esenciales

Lo mínimo que necesitás saber para el TP

Working Directory — `tus archivos` → **Staging Area** — `git add .` → **Repo Local** — `git commit` → **GitHub (remote)** — `git push`

`git init`

Inicializar un repo nuevo en la carpeta actual.

`git clone <url>`

Clonar un repositorio remoto al equipo.

`git status`

Ver qué archivos cambiaron y cuáles están en staging.

`git add .`

Agregar todos los cambios al staging area.

`git commit -m "msg"`

Confirmar los cambios con un mensaje descriptivo.

`git push`

Subir los commits al repositorio remoto (GitHub).

`git pull`

Traer los cambios del repo remoto a la copia local.

`git log --oneline`

Ver el historial de commits de forma resumida.

Tarea de cierre — Primer commit del TP grupal

Antes de irse, cada equipo debe tener su repositorio listo en GitHub

1 Crear el repositorio en GitHub

```
github.com → New repository → nombre: tp-leng1-equipo-N
```

2 Clonar el repo en tu máquina

```
git clone https://github.com/tu-usuario/tp-leng1-equipo-N.git
```

3 Agregar los archivos del TP

```
Copiar catalogo.xml o crear un README.md con los nombres del
equipo
```

4 Hacer el primer commit

```
git add .
git commit -m "feat: estructura inicial del TP"
```

5 Subir a GitHub

```
git push
```

6 Agregar a los compañeros

```
GitHub → Settings → Collaborators → Add people (usuario de
cada integrante)
```

¿Qué aprendimos hoy?

- Los lenguajes de marcado usan etiquetas para estructurar y dar significado al contenido.
- XML es ideal para intercambio de datos entre sistemas — etiquetas personalizables, validación con XSD.
- JSON es más compacto y nativo en la web — estándar de facto para APIs REST modernas.
- La elección entre XML y JSON depende del contexto: interoperabilidad vs. ligereza.
- Git registra el historial de cambios y permite colaborar en equipo sin conflictos.
- Los commits son evidencia de avance — mensajes descriptivos y commits frecuentes.

Próxima clase (Clase 2 — Virtual)

Teoría HTML: estructura de documentos, etiquetas esenciales, semántica y accesibilidad.

Commit Obligatorio 1: Clase 4 · Recordar configurar Git con nombre y email